Experiment No.10

Aim: Explore and implement Autoencoders for image denoising.

Theory:

1. Dataset Source

Dataset: MNIST Dataset

Source link:
https://www.kaggle.com/datasets/hojjatk/mnist-dataset

2. Dataset Description

The MNIST dataset is a benchmark dataset widely used in image processing and deep learning tasks.

Key Characteristics:

- Total samples:
  - Training: 60,000 images
  - Testing: 10,000 images
- Image size: 28×28 pixels
- Type: Grayscale images

Features:

- Pixel values ranging from 0 to 255
- Each image represents a handwritten digit (0–9)

Target Variable:

- In autoencoders, the target is the original clean image itself
- This is a self-supervised learning task

Special Property for this Experiment:

- Artificial Gaussian noise is added to images
- Model learns to reconstruct clean images from noisy inputs

3. Mathematical Formulation of Autoencoder

An autoencoder consists of two main components:

Encoder Function

$$h = f(x) = \sigma(Wx + b)$$

- Compresses input into latent representation

Decoder Function

$$x' = g(h) = \sigma(W'h + b')$$

- Reconstructs original input from encoded data

Objective Function (Loss)

$$L(x, x') = ||x - x'||^2$$

4. Algorithm Limitations

- Autoencoders are data-specific (poor generalization across domains)
- Reconstruction may be blurry due to averaging effect
- Sensitive to bottleneck size
- Requires large data for good performance
- Cannot generate completely new data (unlike GANs)
- Performance depends on noise type

5. Methodology / Workflow

Steps followed:

1. Load MNIST dataset

2. Normalize pixel values to [0,1]
3. Reshape images for CNN input
4. Add Gaussian noise to images
5. Build convolutional autoencoder:
    - Encoder:
        - Conv2D + MaxPooling
    - Decoder:
        - Conv2D + UpSampling
6. Compile model using:
    - Optimizer: Adam
    - Loss: Binary Crossentropy
7. Train model:
    - Input: Noisy images
    - Output: Clean images
8. Evaluate model by:
    - Visual comparison
    - Loss graph
9. Display:
    - Original vs Noisy vs Denoised images
10. Compute reconstruction error
11. Plot histogram of errors

6. Performance Analysis

Evaluation Methods:

- Visual comparison (primary method)
- Reconstruction error (MSE)
- Training & validation loss

Interpretation:

- Autoencoder learns compressed representation
- Effectively separates noise from signal
- Lower reconstruction error indicates better denoising

7. Hyperparameter Tuning

Parameters Tuned:

Parameter	Values Tested
Noise Factor	0.2, 0.4, 0.6
Filters	16, 32, 64
Kernel Size	(3×3)
Epochs	5–10
Batch Size	64, 128

Final Configuration:

- Noise factor: 0.4
- Filters: 32 → 16
- Epochs: 5
- Batch size: 128

Impact of Tuning:

- Higher noise → better robustness
- More filters → better feature extraction
- Drop in loss indicates improved learning
- Balanced architecture avoids overfitting

Source Code:

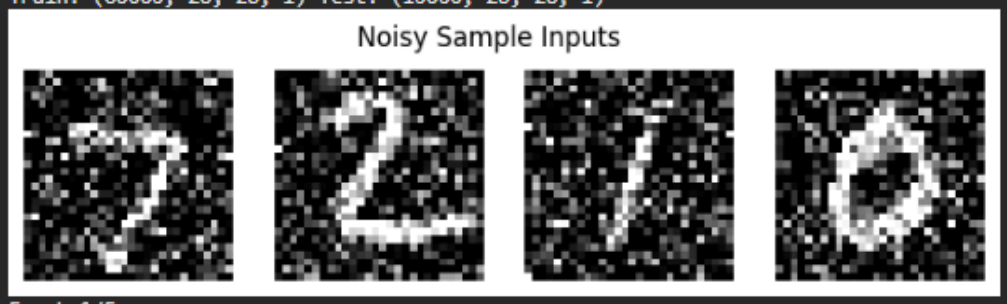
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist

# STEP 2: LOAD DATASET (MNIST)
(x_train, _), (x_test, _) = mnist.load_data()
# Normalize
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
# Reshape
x_train = np.expand_dims(x_train, -1)
x_test = np.expand_dims(x_test, -1)
print("Train:", x_train.shape, "Test:", x_test.shape)
```

```
# STEP 3: ADD NOISE
noise_factor = 0.4
x_train_noisy = np.clip(x_train + noise_factor * np.random.normal(size=x_train.shape), 0., 1.)
x_test_noisy = np.clip(x_test + noise_factor * np.random.normal(size=x_test.shape), 0., 1.)
# STEP 4: SHOW NOISY IMAGES
plt.figure(figsize=(8,2))
for i in range(4):
    plt.subplot(1,4,i+1)
    plt.imshow(x_test_noisy[i].squeeze(), cmap='gray')
    plt.axis('off')
plt.suptitle("Noisy Sample Inputs")
plt.show()
# STEP 5: BUILD AUTOENCODER (CNN)
input_img = layers.Input(shape=(28,28,1))
# Encoder
x = layers.Conv2D(32, (3,3), activation='relu', padding='same')(input_img)
x = layers.MaxPooling2D((2,2), padding='same')(x)
x = layers.Conv2D(16, (3,3), activation='relu', padding='same')(x)
encoded = layers.MaxPooling2D((2,2), padding='same')(x)
# Decoder
x = layers.Conv2D(16, (3,3), activation='relu', padding='same')(encoded)
x = layers.UpSampling2D((2,2))(x)
x = layers.Conv2D(32, (3,3), activation='relu', padding='same')(x)
x = layers.UpSampling2D((2,2))(x)
decoded = layers.Conv2D(1, (3,3), activation='sigmoid', padding='same')(x)
autoencoder = models.Model(input_img, decoded)
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')
# STEP 6: TRAIN MODEL
history = autoencoder.fit(
    x_train_noisy, x_train,
    epochs=5,
    batch_size=128,
    shuffle=True,
    validation_split=0.1
)
# STEP 7: LOSS GRAPH
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title("Autoencoder Training Loss")
plt.legend()
```

```
plt.show()

# STEP 8: DENOISED OUTPUT
decoded_imgs = autoencoder.predict(x_test[:10])

plt.figure(figsize=(15,5))
for i in range(10):
    # Original
    ax = plt.subplot(3,10,i+1)
    plt.imshow(x_test[i].squeeze(), cmap='gray')
    plt.axis('off')
    if i==0: ax.set_ylabel("Original")
    # Noisy
    ax = plt.subplot(3,10,i+11)
    plt.imshow(x_test_noisy[i].squeeze(), cmap='gray')
    plt.axis('off')
    if i==0: ax.set_ylabel("Noisy")
    # Denoised
    ax = plt.subplot(3,10,i+21)
    plt.imshow(decoded_imgs[i].squeeze(), cmap='gray')
    plt.axis('off')
    if i==0: ax.set_ylabel("Denoised")
plt.show()

# STEP 9: ERROR TABLE
recon_error = np.mean((decoded_imgs - x_test[:10])**2, axis=(1,2,3))

error_table = pd.DataFrame({
    'Sample': np.arange(1,11),
    'ReconstructionMSE': recon_error
})

print("\nError Table:")
print(error_table)
# STEP 10: HISTOGRAM (PDF PAGE 10)
plt.hist(recon_error, bins=10)
plt.title("Reconstruction Error Distribution")
plt.xlabel("MSE")
plt.ylabel("Frequency")
plt.show()
```

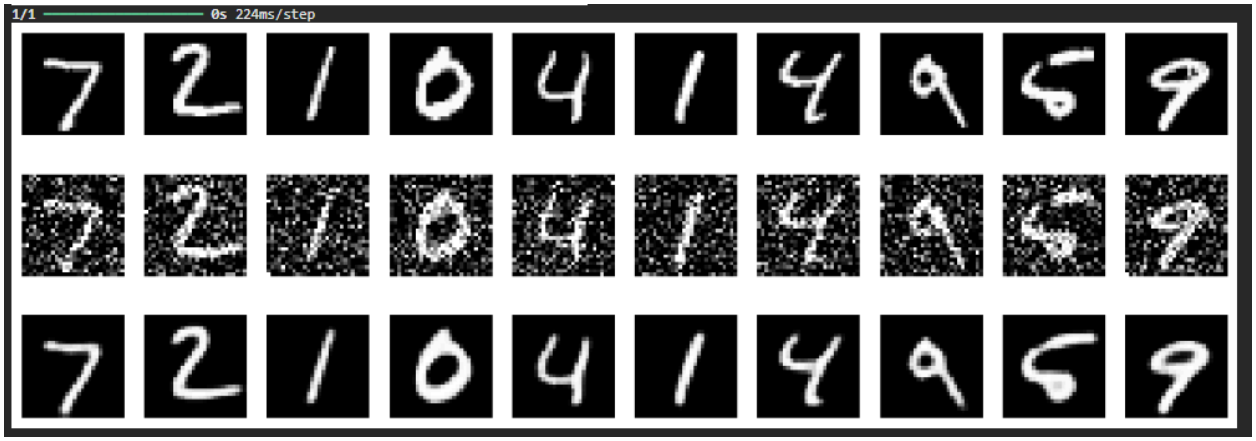
Output:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Train: (60000, 28, 28, 1) Test: (10000, 28, 28, 1)
```

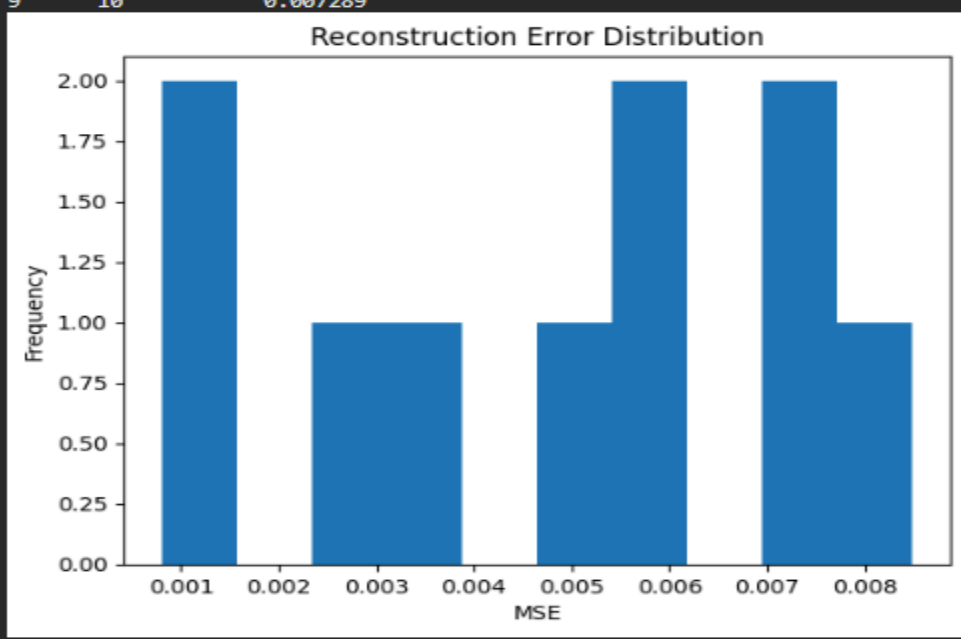


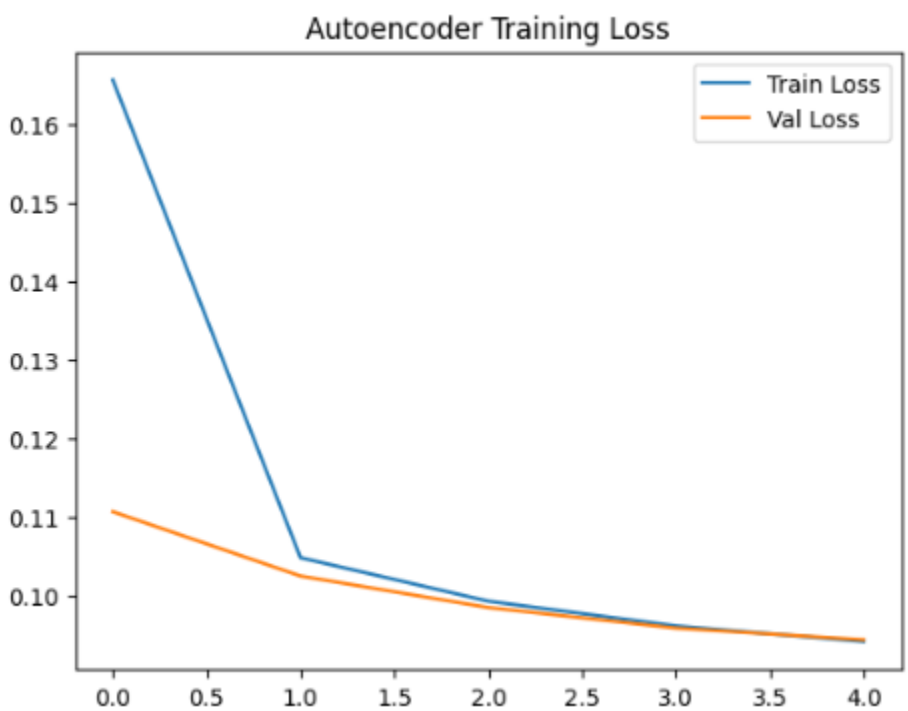

```
Epoch 1/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 105s 244ms/step - loss: 0.1657 - val_loss: 0.1107
Epoch 2/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 101s 238ms/step - loss: 0.1049 - val_loss: 0.1025
Epoch 3/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 100s 236ms/step - loss: 0.0993 - val_loss: 0.0985
Epoch 4/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 141s 233ms/step - loss: 0.0962 - val_loss: 0.0959
Epoch 5/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 100s 238ms/step - loss: 0.0942 - val_loss: 0.0944
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 224ms/step
```



```
Error Table:
   Sample  ReconstructionMSE
0       1           0.003091
1       2           0.003221
2       3           0.001145
3       4           0.005549
4       5           0.005347
5       6           0.000797
6       7           0.007107
7       8           0.005967
8       9           0.008492
9      10           0.007289
```





Conclusion:

The convolutional autoencoder effectively removes noise from handwritten digit images by learning meaningful latent representations. The model demonstrates strong capability in reconstructing clean images from noisy inputs. Proper architecture design, noise handling, and hyperparameter tuning significantly improve denoising performance. Autoencoders are powerful tools for unsupervised learning tasks such as image denoising and feature extraction.